INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

# Tecnologías de desarrollo en el servidor




## Entrega 2: Autenticación y autorización (documento v3)

Presentan

Emilio Daniel Guzmán Seda

Daniel Sebastián Macías Macías

Profesor: Francisco Javier Sevilla Medina

Fecha: jueves 5 de noviembre de 2026

# Proyecto Integrador — Recall

**Documento del proyecto · versión 3** · Entrega 2: Autenticación y autorización · 5 de noviembre de 2026

Recall es el REST API de una plataforma web que recibe audio de reuniones, clases y entrevistas, lo transcribe y genera resúmenes con IA, sin depender de un dispositivo dedicado. El MVP se mantiene lo más simple posible: un solo API en TypeScript con Express, PostgreSQL en Docker, autenticación propia con JWT, login opcional con Google y servicios de AWS (S3, Transcribe y Bedrock) para las siguientes entregas.

## Cambios respecto a la versión 2

| Tema | Versión 2 | Versión 3 (esta entrega) |
|---|---|---|
| Autenticación | Amazon Cognito, con un modo de prueba sin verificación | Autenticación propia: contraseñas con bcrypt y sesión con JWT firmado (HS256). Se eliminó Cognito |
| Modelo User | `cognitoSub` (único) | `passwordHash` (bcrypt, nulo para cuentas de Google) y `role` |
| Login con proveedor | Planeado con el Hosted UI de Cognito | Google con OAuth 2.0 / OpenID Connect, implementado en el propio API |
| Roles y permisos | Solo documentados | `USER` y `ADMIN` implementados en el API y en las vistas, con revisión del dueño de cada recurso |
| Vistas | Listado y detalle públicos | Registro, login con sesión en cookie, y cada persona ve solo lo que le corresponde |
| Colección de Postman | 66 peticiones | 95 peticiones, con carpetas de Permisos y casos de seguridad |
| Documento | Dos documentos (proyecto y entrega 2) | Un solo documento con las evidencias de las pruebas |

## Datos generales

| Campo | Valor |
|---|---|
| Nombre del proyecto | Recall |
| Tipo | REST API con TypeScript y Express, con vistas en Handlebars |
| Repositorio | github.com/sebastian0023/Recall (privado) |
| Acceso del profesor | fsevilla@gmail.com como colaborador |
| Materia | Tecnologías de desarrollo en el servidor · ITESO |

### Integrantes

| Nombre | Responsabilidades |
|---|---|
| Daniel Sebastián Macías Macías | Conexión a la BD, modelos y migraciones, CRUD con Prisma, validaciones con zod, manejo de errores, autenticación con JWT y bcrypt, y login con Google |
| Emilio Daniel Guzmán Seda | Seed, vistas con Handlebars (incluido el registro), roles y permisos, colección de Postman y casos de seguridad, CI/CD (ESLint, SonarQube, Docker) y documento |

Ambos integrantes trabajan en ramas por funcionalidad que llegan a `main` mediante Pull Request.

## Descripción

### Problema

En reuniones, clases y entrevistas se pierde información porque tomar notas y prestar atención al mismo tiempo es difícil. Las herramientas que lo resuelven suelen requerir hardware de pago, están pensadas para el inglés o cobran suscripciones en dólares.

### Solución propuesta

Recall permite subir un archivo de audio. El API lo guarda en Amazon S3, lo transcribe con Amazon Transcribe y genera un resumen con Amazon Bedrock. El usuario consulta y edita sus grabaciones, transcripciones y resúmenes desde el API o desde las vistas web.

### Alcance del MVP

- Registro e inicio de sesión con correo y contraseña, o con Google.
- Roles `USER` y `ADMIN`: cada usuario administra solo sus datos; el administrador ve todo.
- CRUD de grabaciones; el archivo de audio se guarda en S3.
- Transcripción en español con Amazon Transcribe.
- Resumen con Amazon Bedrock usando una de tres plantillas fijas: reunión, clase o entrevista.
- Consulta y edición de transcripciones y resúmenes, desde el API y desde vistas web.

### Esta entrega (Autenticación y autorización)

- Registro y login propios: la contraseña se guarda con bcrypt y la sesión es un JWT firmado con HS256.
- Login con Google (OAuth 2.0 / OpenID Connect) que abre la misma sesión que el login con contraseña.
- Roles `USER` y `ADMIN` con una matriz de permisos aplicada en el API y en las vistas.
- Vistas de registro y login, con la sesión guardada en una cookie `httpOnly`.
- Colección de Postman con los casos de seguridad: tokens alterados, sin token, intento de asignarse un rol y respuestas que no revelan información.

### Fuera del alcance

- Carpetas, etiquetas y plantillas personalizadas.
- Transcripción en tiempo real del audio, aplicación móvil y compartir grabaciones.
- Recuperación de contraseña y verificación de correo para cuentas con contraseña.
- Despliegue en la nube: el proyecto corre en local (la imagen Docker queda lista para un despliegue futuro).

## Proyectos similares y referencias

Recall toma como referencia principal a Plaud, pero elimina la dependencia de hardware y se enfoca en usuarios hispanohablantes.

| Referencia | Qué ofrece | Diferencia con Recall |
|---|---|---|
| Plaud | Grabadoras físicas (Note, Note Pro, NotePin) de USD 159 a 189, con app que transcribe y resume con plantillas. Plan Pro de USD 17.99 al mes para 1,200 minutos. | Recall no requiere comprar un dispositivo: graba desde el navegador. |
| Otter.ai | Transcripción de reuniones y audios importados en 6 idiomas, con resúmenes y bot para Zoom, Teams y Meet. Plan gratuito limitado a 3 archivos importados. | Recall prioriza archivos subidos y grabación presencial, sin límite artificial de importaciones. |
| Fireflies.ai, tl;dv | Bots que se unen a videollamadas, transcriben y generan notas. | Enfocados en reuniones en línea; Recall cubre clases y entrevistas presenciales. |
| Grabadora de Google Pixel | Transcripción local en el teléfono. | Exclusiva de un modelo de teléfono y sin resúmenes por plantilla. |

### Motivo para desarrollar una herramienta similar

- **Sin hardware:** cualquier persona con navegador puede usarla, sin pagar un dispositivo de más de USD 150.
- **Español latino como prioridad:** plantillas y resúmenes pensados para contextos académicos y laborales en México.
- **Control de los datos:** el audio vive en un bucket propio, con políticas de retención definidas por el proyecto.
- **Fin académico:** integra autenticación, base de datos relacional, almacenamiento en la nube, contenedores y patrones de diseño.

# Stack de tecnologías

## Backend y base de datos

| Capa | Tecnología | Uso en el proyecto |
|---|---|---|
| Lenguaje | TypeScript (strict) | Tipado estático |
| Runtime | Node.js 24 LTS | Ejecución del API |
| Framework HTTP | Express 5 | Rutas, middlewares, controladores y vistas |
| Base de datos | PostgreSQL 17 | Usuarios, grabaciones, transcripciones y resúmenes |
| ORM y migraciones | Prisma 7 | Esquema, migraciones versionadas y consultas tipadas |
| Validación | zod | Esquemas de body, params y query |
| Autenticación | bcryptjs y jsonwebtoken | Hash de contraseñas y sesión con JWT (HS256) |
| Login con proveedor | Google OAuth 2.0 / OpenID Connect | Login opcional con una cuenta de Google |
| Vistas | Handlebars (express-handlebars) | Registro, login, listados y detalle con datos de la BD |
| Contenedores | Docker Compose / Dockerfile | PostgreSQL local e imagen del API |
| Calidad y CI/CD | ESLint, SonarQube Cloud, GitHub Actions | Lint, quality gate, pruebas de la colección e imagen Docker en cada push |

## Librerías

| Librería | Para qué se usa | Entrega |
|---|---|---|
| express, cors, dotenv | Servidor HTTP, orígenes permitidos y variables de entorno | Planeación |
| prisma, @prisma/client, @prisma/adapter-pg, pg | Esquema, migraciones y conexión a PostgreSQL | Planeación / Entrega 1 |
| zod | Validación de body, params y query | Entrega 1 |
| express-handlebars | Motor de vistas | Entrega 1 |
| bcryptjs | Hash de contraseñas (10 rondas) | Entrega 2 (esta) |
| jsonwebtoken | Firma y verificación de los JWT | Entrega 2 (esta) |
| cookie-parser | Leer la cookie de sesión de las vistas | Entrega 2 (esta) |
| eslint, typescript-eslint | Linter de TypeScript (también corre en CI) | Entrega 1 |
| tsx, typescript | Ejecución en desarrollo, seed y compilación | Planeación |
| multer, @aws-sdk/client-s3 | Recibir el audio y subirlo a S3 | Siguiente entrega |
| @aws-sdk/client-transcribe, client-bedrock-runtime | Transcribir y resumir | Siguientes entregas |

El login con Google no usa una librería: el API habla directamente con los endpoints de autorización y de token de Google con `fetch`.

## Servicios externos

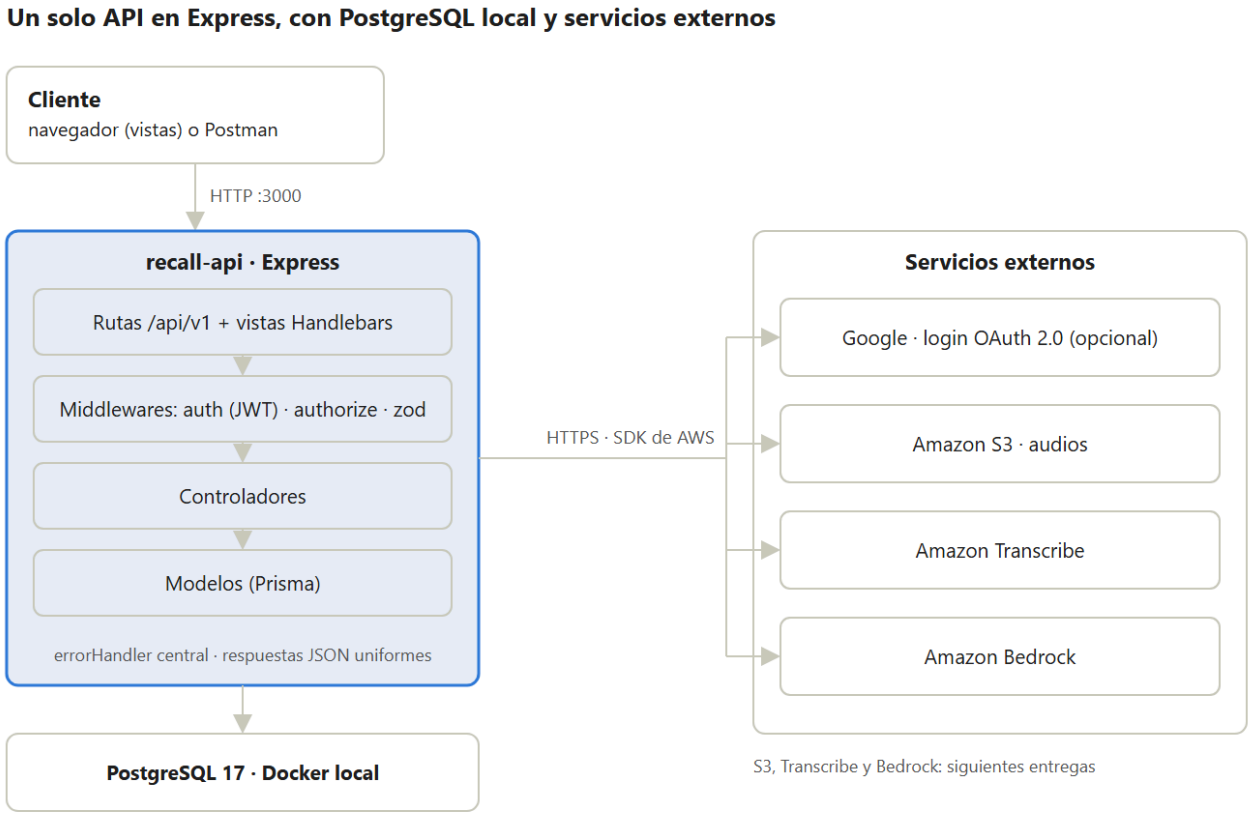
| Servicio | Uso |
|---|---|
| Google (OAuth 2.0 / OpenID Connect) | Login opcional con una cuenta de Google. Se activa con tres variables de entorno |
| Amazon S3 | Guardar los archivos de audio en un bucket privado (siguiente entrega) |
| Amazon Transcribe | Transcripción por lotes en español (es-US o es-MX) con separación de hablantes |
| Amazon Bedrock | Generar el resumen con un modelo de lenguaje a partir de la plantilla elegida |

## Alternativas investigadas

| Necesidad | Elegida | Alternativas |
|---|---|---|
| Framework | Express | NestJS, Fastify |
| ORM | Prisma | TypeORM, Drizzle |
| Validación | zod | express-validator, joi |
| Vistas | Handlebars | EJS, Pug, cliente React separado |
| Autenticación | JWT propio con bcrypt | Amazon Cognito, Auth0, sesiones en servidor |
| Login con proveedor | Google con OAuth 2.0 directo | Passport.js, Cognito Hosted UI |
| Transcripción | Amazon Transcribe | OpenAI Whisper, Deepgram, AssemblyAI |
| Resúmenes | Amazon Bedrock | OpenAI API, Ollama local |
| Almacenamiento | Amazon S3 | Cloudflare R2, MinIO |

En la versión 2 se usaba Amazon Cognito. Se cambió a autenticación propia porque la entrega pide implementar el registro, el login y el manejo de tokens en el API, y porque Cognito dependía de una sola cuenta de AWS para probar cualquier cosa. Con JWT propio, cualquier integrante (y el profesor) puede correr y probar todo en local.

# Arquitectura




Arquitectura del MVP · un API en Express, PostgreSQL local y servicios externos

El API corre en la máquina del desarrollador y se conecta a PostgreSQL en Docker. Sigue el patrón MVC:

- **Rutas** (`src/routes/`): definen los endpoints de cada módulo bajo `/api/v1` y las rutas de las vistas, y aplican los middlewares.
- **Middlewares** (`src/middlewares/`): `requireAuth` y `viewAuth` (verifican el JWT), `requireRole` y `requireSelfOrAdmin` (permisos), `validate` (zod), `notFound` (404) y `errorHandler` (respuesta de error uniforme).
- **Controladores** (`src/controllers/`): reciben la petición validada, revisan permisos, consultan la BD con Prisma y responden JSON o renderizan una vista.
- **Modelos** (`prisma/schema.prisma` y `src/models/`): esquema de la BD y tipos de entrada de cada entidad.
- **Vistas** (`views/`): plantillas Handlebars con un layout base y navegación.

## Flujo de procesamiento (siguientes entregas)

- `POST /recordings` recibe el audio, lo sube a S3 y crea la grabación con estado UPLOADED.
- `POST /recordings/:id/process` inicia un trabajo en Transcribe y cambia el estado a PROCESSING (hoy solo cambia el estado).

- Cuando el trabajo termina, el API guarda la transcripción, pide el resumen a Bedrock, lo guarda y cambia el estado a DONE (o FAILED si algo falla).

## Base de datos

La base de datos es relacional (PostgreSQL 17) con 4 tablas definidas en `prisma/schema.prisma`. Las llaves primarias son UUID y las 5 migraciones están versionadas en `prisma/migrations`. El diagrama se actualizó: User ya no tiene `cognitoSub` y ahora tiene `passwordHash` y `role`.

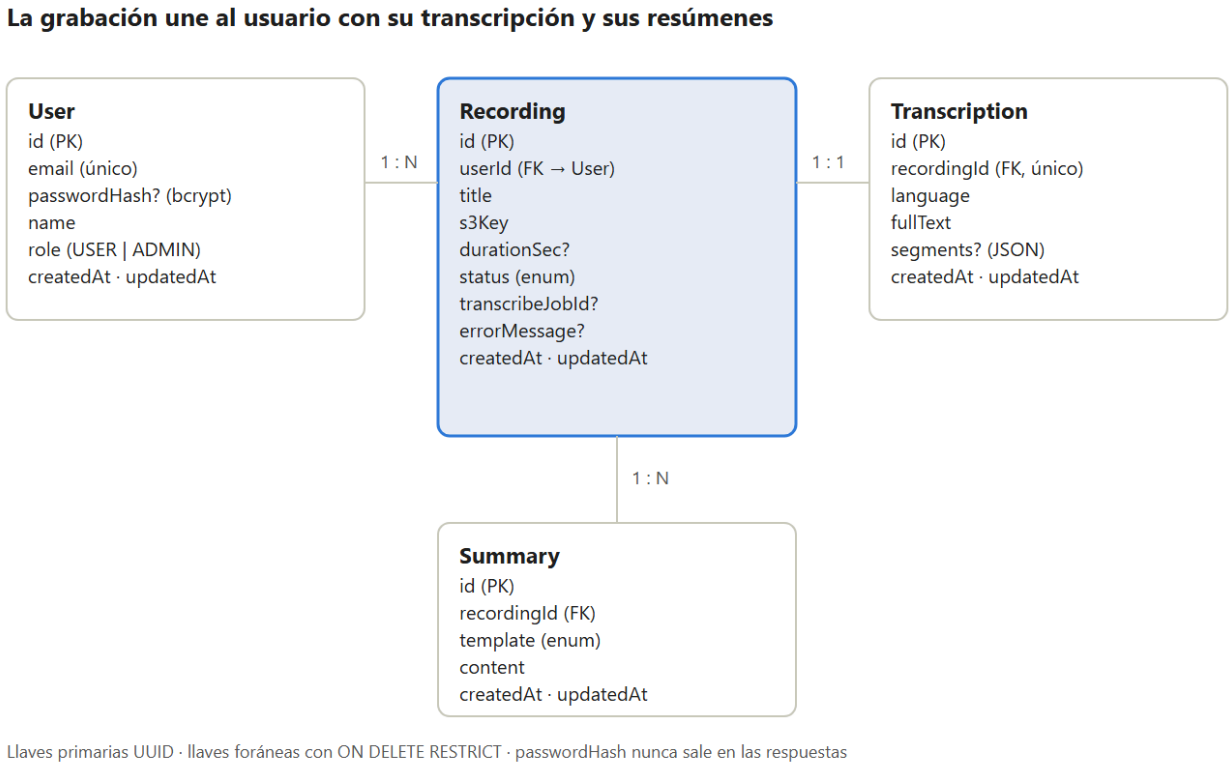



Diagrama entidad-relación · 4 tablas en PostgreSQL

## Modelo User

| Campo | Tipo | Notas |
|---|---|---|
| `id` | UUID | Llave primaria |
| `email` | texto, único | Duplicado responde 409 |
| `passwordHash` | texto, opcional | Hash bcrypt de 10 rondas. Nulo para cuentas creadas con Google. Prisma lo omite en todas las consultas, así que nunca llega a una respuesta |
| `name` | texto | Hasta 100 caracteres |
| `role` | enum USER · ADMIN | USER por defecto. Nunca se toma del registro ni de Google |

| Campo | Tipo | Notas |
|---|---|---|
| `createdAt`, `updatedAt` | fecha | Automáticos |

### Relaciones

| Relación | Cardinalidad | Explicación |
|---|---|---|
| User → Recording | 1 : N | Un usuario tiene muchas grabaciones; cada grabación pertenece a un solo usuario (`userId`). |
| Recording → Transcription | 1 : 1 | Una grabación tiene a lo más una transcripción; `recordingId` es único. |
| Recording → Summary | 1 : N | Una grabación puede tener varios resúmenes, por ejemplo uno por plantilla. |

### Índices y reglas

- Índices únicos: `User.email` y `Transcription.recordingId`. Un duplicado responde 409.
- Índices de búsqueda: `Recording.userId` y `Summary.recordingId`.
- `Recording.status` es un enum: UPLOADED, PROCESSING, DONE, FAILED. `Summary.template` es un enum: MEETING, CLASS, INTERVIEW. `User.role` es un enum: USER, ADMIN.
- Los fragmentos por hablante se guardan en un campo JSON (`segments`) para no crear una tabla extra.
- El audio no se guarda en la base de datos, solo su llave en S3 (`s3Key`).

### Política de borrado

Borrado físico resuelto en el código. Las llaves foráneas son RESTRICT, así que la base de datos nunca borra en cascada por su cuenta:

| Se elimina | Qué pasa |
|---|---|
| Recording | Se borran sus resúmenes y su transcripción y después la grabación, en una transacción. Responde 204. |
| User | Si tiene grabaciones no se elimina: responde 409. Primero hay que borrar sus grabaciones. |
| Transcription / Summary | Se eliminan directamente (no tienen dependientes). Responde 204. |

## Autenticación

### Registro y login con contraseña

- `POST /api/v1/auth/register` recibe `name`, `email` y `password` (8 a 72 caracteres). El esquema es estricto: cualquier otro campo, como `role`, responde 400. El usuario siempre se crea con rol USER.

- La contraseña se guarda con **bcrypt** (10 rondas). Nunca se guarda en texto plano y nunca aparece en una respuesta.
- `POST /api/v1/auth/login` devuelve `{ accessToken, tokenType: "Bearer", expiresIn, user }`.
- **Mismo mensaje en el login:** si el correo no existe o la contraseña es incorrecta, responde lo mismo, `401 Incorrect email or password`. Además compara siempre contra un hash bcrypt, aunque el correo no exista, para que el tiempo de respuesta tampoco revele qué correos están registrados.

## Sesión con JWT

| Aspecto | Cómo se hace |
|---|---|
| Algoritmo | HS256, firmado con `JWT_SECRET` (mínimo 32 caracteres). Solo se acepta HS256, así que un token con `alg: none` se rechaza |
| Contenido | Solo `sub` (id del usuario), `iat` y `exp`. El rol no va en el token: se lee de la BD en cada petición, así un cambio de rol surte efecto de inmediato |
| Vigencia | `JWT_EXPIRES_IN` (por defecto `1h`). Un token vencido responde `401 Token expired` |
| API | `Authorization: Bearer <token>`. Sin token responde `401 Unauthorized`; un token alterado o firmado con otra clave, `401 Invalid token` |
| Vistas | El mismo JWT en la cookie `recall_session`, con `httpOnly`, `sameSite=lax` y `secure` en producción. Las rutas del API no aceptan la cookie, solo el header, para evitar ataques CSRF |
| Producción | El API se niega a arrancar con el `JWT_SECRET` de ejemplo cuando `NODE_ENV=production` |

## Login con Google

Flujo OAuth 2.0 / OpenID Connect con código de autorización, implementado sin librerías externas:

1. `GET /auth/google` genera un `state` y un `nonce` aleatorios, los guarda en una cookie `httpOnly` de 10 minutos y redirige a Google.
2. Google regresa a la URL de `GOOGLE_CALLBACK_URL` (por defecto `/google_confirm`). El API revisa que el `state` coincida con el de la cookie, cambia el código por el `id_token` y valida el emisor, la audiencia, la vigencia, el `nonce` y que el correo esté **verificado** por Google.
3. Si ya existe un usuario con ese correo, entra a esa cuenta; si no, la crea sin contraseña y con rol `USER`. El rol nunca viene de Google.
4. Abre la misma sesión que el login con contraseña (JWT en la cookie), así que los permisos son idénticos.

Es opcional: sin `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` y `GOOGLE_CALLBACK_URL` el botón «Continuar con Google» no aparece. Las credenciales solo viven en el `.env`.

## Roles y permisos

| Rol | Descripción |
|---|---|
| Invitado | Persona sin sesión. Solo puede registrarse, iniciar sesión y consultar `/health`. |
| Usuario (USER) | Rol por defecto al registrarse. Administra únicamente sus propias grabaciones, transcripciones y resúmenes. |
| Administrador (ADMIN) | Soporte y moderación: ve y administra los recursos de todos los usuarios y puede eliminar usuarios. |

El rol solo lo cambia un administrador con `PATCH /users/:id` y `{ "role": "ADMIN" }`. El primer administrador viene del seed (Sebastián Macías) o se asigna directamente en la base de datos.

### Matriz de permisos

«Propios» significa que la acción solo se permite sobre recursos del mismo usuario; sobre los de otro responde 403.

| Acción | Invitado | Usuario | Admin | Dónde se aplica |
|---|---|---|---|---|
| Registrarse e iniciar sesión | Sí | Sí | Sí | Rutas públicas |
| Ver su perfil (`GET /auth/me`) | No | Sí | Sí | `requireAuth` |
| Listar usuarios | No | No | Sí | `requireRole("ADMIN")` |
| Ver o editar usuario | No | Propio | Todos | `requireSelfOrAdmin` |
| Cambiar el rol de un usuario | No | No | Sí | `updateUser` |
| Eliminar usuario | No | No | Sí | `requireRole("ADMIN")` |
| Listar grabaciones | No | Propias | Todas | Filtro en `getRecordings` |
| Crear grabación | No | A su nombre | Para cualquier usuario | `assertCanAccess` |
| Ver, editar, procesar o eliminar grabación | No | Propias | Todas | `findRecordingOr404` |
| Transcripciones y resúmenes | No | De sus grabaciones | Todos | `assertCanAccess` |
| Vistas web | No | Sus datos | Todos los datos | `viewAuth` y los mismos filtros |

Orden de las respuestas: primero 401 (sin sesión válida) y 400 (datos inválidos). En grabaciones, transcripciones y resúmenes sigue 404 (no existe) y luego 403 (no es tuyo). En `/users/:id`, un usuario normal recibe 403 con cualquier id que no sea el suyo, exista o no.

# REST API

Todas las rutas usan el prefijo `/api/v1` (excepto `/health` y las vistas). Las respuestas exitosas tienen la forma `{ data, message }` y los errores `{ error: { message, details? } }`.

## Endpoints

| Módulo | Método y ruta | Acceso | Respuestas |
|---|---|---|---|
| Health | `GET /health` | Público | 200 |
| Auth | `POST /auth/register` | Público | 201 (rol USER); 400; 409 correo duplicado |
| Auth | `POST /auth/login` | Público | 200 con `accessToken`; 400; 401 genérico |
| Auth | `GET /auth/me` | Sesión | 200; 401 |
| Users | `GET /users` | ADMIN | 200; 401; 403 |
| Users | `GET /users/:id` · `PATCH /users/:id` | Propio o ADMIN | 200; 400; 401; 403; 404; 409 correo duplicado |
| Users | `DELETE /users/:id` | ADMIN | 204; 401; 403; 404; 409 con grabaciones |
| Recordings | `GET /recordings` · `GET /recordings/:id` | Propias o ADMIN | 200; 400; 401; 403; 404 |
| Recordings | `GET /recordings/:id/summaries` · `/transcription` | Propias o ADMIN | 200 (rutas anidadas); 401; 403; 404 |
| Recordings | `POST` · `PATCH /:id` · `DELETE /:id` | Propias o ADMIN | 201 / 200 / 204; 400; 401; 403; 404 |
| Recordings | `POST /recordings/:id/process` | Propias o ADMIN | 202; 401; 403; 404 |
| Transcriptions | `GET` · `GET /:id` · `POST` · `PATCH /:id` · `DELETE /:id` | De sus grabaciones o ADMIN | 200 / 201 / 204; 400; 401; 403; 404; 409 si la grabación ya tiene transcripción |
| Summaries | `GET` · `GET /:id` · `POST` · `PATCH /:id` · `DELETE /:id` | De sus grabaciones o ADMIN | 200 / 201 / 204; 400; 401; 403; 404 |

Los listados aceptan filtros validados: `?status=&userId=` en recordings, `?recordingId=&template=` en summaries y `?recordingId=` en transcriptions.

## Validaciones y errores

- El middleware reutilizable `validate({ body, params, query })` corre antes del controlador con esquemas zod: obligatorios, tipos, longitudes, formatos (correo, UUID, código de idioma), rangos y enums. Un campo desconocido también es error.

- Una petición inválida responde 400 e indica en `details` qué campo falló y por qué.
- El `errorHandler` central traduce los errores de Prisma: P2002 → 409, P2025 → 404, P2003 → 409, P2023 → 400. Nunca responde con el stack ni con el código de la BD; ese detalle solo se escribe en la consola del servidor.
- Las rutas inexistentes responden 404 en JSON.

## Vistas

| Ruta | Vista | Acceso |
|---|---|---|
| `GET /register` · `POST /register` | Crear cuenta: nombre, correo y contraseña. Valida cada campo, responde 409 si el correo ya existe, crea un USER, abre la sesión y redirige al inicio | Público |
| `GET /login` · `POST /login` · `POST /logout` | Iniciar y cerrar sesión, con enlace al registro y botón de Google si está configurado | Público |
| `GET /auth/google` · `GET /google_confirm` | Inicio y regreso del login con Google | Público |
| `GET /` | Inicio: conteos y últimas grabaciones | Sesión (sus datos) |
| `GET /recordings` · `GET /recordings/:id` | Listado con filtro por estado y detalle con transcripción y resúmenes | Sesión (propias, o todas si es ADMIN) |
| `GET /users` | Usuarios con su rol y su número de grabaciones | ADMIN |

Sin sesión, las vistas protegidas redirigen a `/login`. La barra muestra el nombre y el rol, y oculta «Usuarios» a quien no es administrador.

## Datos de prueba y colección

### Seed

`npm run seed` (`prisma/seed.ts`) limpia las 4 tablas y vuelve a cargar los mismos registros en una transacción, por lo que se puede correr varias veces sin errores ni duplicados: 6 usuarios, 8 grabaciones (con los 4 estados), 5 transcripciones y 6 resúmenes de las 3 plantillas. Todos los usuarios usan la contraseña `Recall2026!`, guardada con bcrypt. Sebastián Macías es `ADMIN`; los demás, `USER`.

### Colección de Postman

`docs/api.postman_collection.json` tiene **95 peticiones** en 8 carpetas: Health, Auth, Users, Recordings, Transcriptions, Summaries, Permisos y Errores generales. La URL base está en la variable `{{baseUrl}}`, y los tokens de admin y de usuario se guardan solos al correr la carpeta Auth.

| Caso de seguridad | Respuesta esperada |
|---|---|
| Registro con `role: "ADMIN"` en el body | 400, indica el campo `role` |
| Login sin contraseña | 400, indica el campo `password` |
| Contraseña incorrecta y correo inexistente | 401 con exactamente la misma respuesta |
| La contraseña nunca aparece en las respuestas | Registro, login, perfil y usuarios no contienen `password` |
| Sin token (perfil, crear grabación, eliminar usuario y listados) | 401 `Unauthorized` |
| Token alterado: `sub` cambiado al del admin | 401 `Invalid token` |
| Token alterado: firma modificada | 401 `Invalid token` |
| Token sin firma (`alg: none`) | 401 `Invalid token` |
| Token firmado con otra clave | 401 `Invalid token` |
| Usuario intenta acciones de otro o de admin | 403 |

Los tokens alterados se generan en el *pre-request* a partir de un token real, así que la prueba siempre ataca un token válido del mismo servidor. El **token expirado** no se puede automatizar en Postman sin esperar a que venza, así que se prueba a mano con `JWT_EXPIRES_IN` corto (ver Evidencia 4).

## Pruebas y evidencia

Las pruebas se corrieron el 5 de noviembre de 2026 en una base PostgreSQL 17 desechable, con las 5 migraciones aplicadas y los datos del seed. Las imágenes de terminal son la salida real de los comandos, y las de las vistas son capturas del navegador con la aplicación en ejecución.

### Calidad y migraciones

Lint, typecheck, migraciones, seed y build

```
$ npm run lint
OK sin errores

$ npm run typecheck
OK sin errores

$ npx prisma migrate deploy
5 migrations found in prisma/migrations
No pending migrations to apply.

$ npx prisma migrate diff --from-config-datasource --to-schema prisma/schema.prisma
No difference detected.

$ npm run seed
Seed listo: 6 users, 8 recordings, 5 transcriptions, 6 summaries

$ npm run build
OK compilado en dist/
```

Evidencia 1 · ESLint y TypeScript sin errores; la base queda idéntica a `schema.prisma`

## Colección de Postman

Corrieron las 95 peticiones con newman: 163 aserciones y ninguna falla. La imagen muestra los casos de seguridad y el resumen de toda la colección.

Colección de Postman: casos de seguridad y resumen de las 95 peticiones

```
$ npx newman run docs/api.postman_collection.json

OK  Registrar - intenta asignarse role ADMIN (400) -> 400 (3 pruebas)
OK  Login (admin, guarda adminToken) -> 200 (3 pruebas)
OK  Login - usuario normal (guarda userToken) -> 200 (3 pruebas)
OK  Login - contraseña incorrecta (401) -> 401 (2 pruebas)
OK  Login - usuario inexistente (401) -> 401 (3 pruebas)
OK  Login - sin contraseña (400) -> 400 (2 pruebas)
OK  Usuario autenticado (me) -> 200 (2 pruebas)
OK  Sin token (401) -> 401 (1 pruebas)
OK  Token inválido (401) -> 401 (1 pruebas)
OK  Token alterado: cambia el sub al del admin (401) -> 401 (2 pruebas)
OK  Token alterado: firma modificada (401) -> 401 (2 pruebas)
OK  Token sin firma, alg none (401) -> 401 (2 pruebas)
OK  Token firmado con otra clave (401) -> 401 (2 pruebas)
OK  Sin token - su perfil (401) -> 401 (1 pruebas)
OK  Sin token - crear grabación (401) -> 401 (1 pruebas)
OK  Sin token - eliminar usuario (401) -> 401 (1 pruebas)
OK  Usuario - listar usuarios (403) -> 403 (1 pruebas)
OK  Usuario - cambiarse el rol (403) -> 403 (1 pruebas)
OK  Usuario - ver grabación ajena (403) -> 403 (1 pruebas)
OK  Admin - ver grabación de otro usuario -> 200 (1 pruebas)

Resumen de toda la colección
  requests            ejecutadas:   95   fallidas: 0
  test-scripts        ejecutadas:   95   fallidas: 0
  prerequest-scripts  ejecutadas:    4   fallidas: 0
  assertions          ejecutadas:  163   fallidas: 0
```

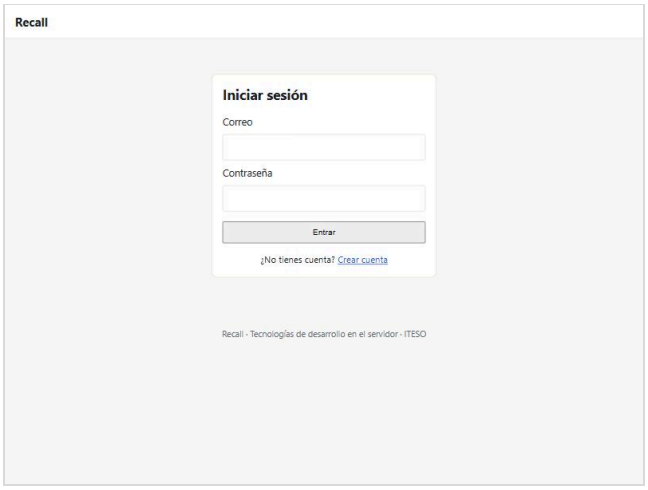
Evidencia 2 · Casos de seguridad de la colección y resumen de las 95 peticiones

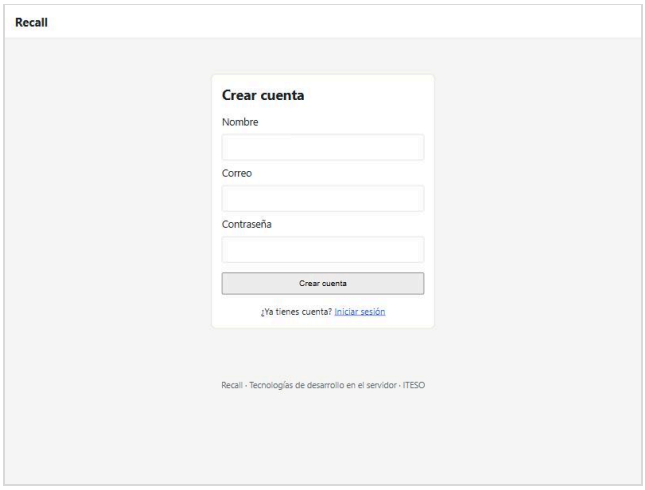
## Vista de registro

Vista de registro probada con curl

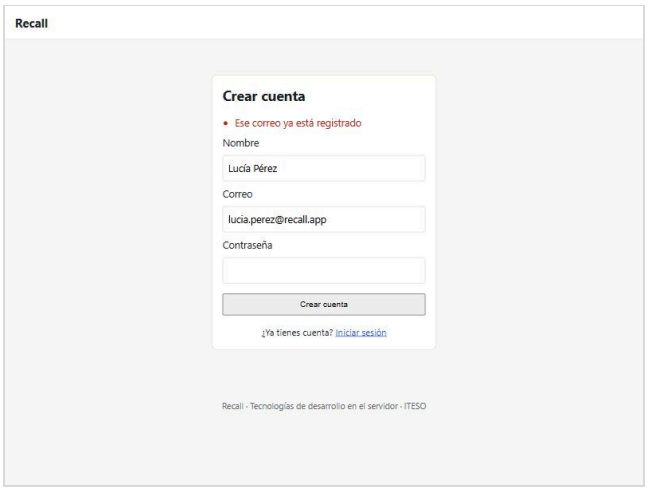
```
Vista de registro (POST /register)
GET /register                                          -> 200
Campos válidos                                        -> 302 http://localhost:3993/
Con la cookie recibida: GET /recordings                -> 200
Mismo correo otra vez                                  -> 409 Ese correo ya está registrado
Correo inválido y contraseña corta                    -> 400 Escribe un correo válido La contraseña debe tener entre 8 y 72 caracteres
Formulario alterado con role=ADMIN                     -> 400 El formulario tiene campos no permitidos
```

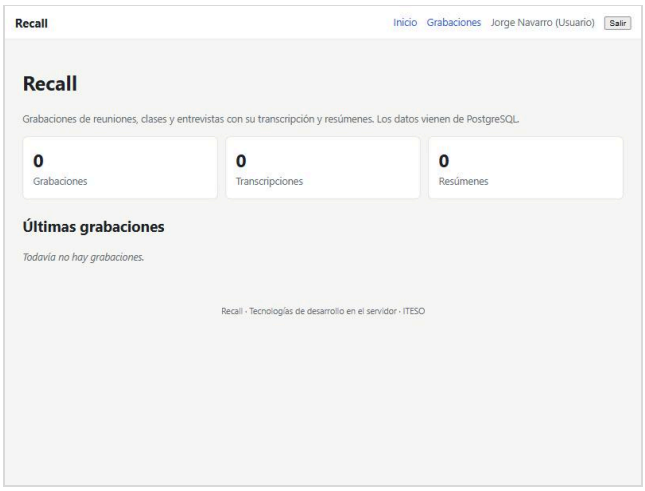
Evidencia 3 · El registro crea la cuenta e inicia sesión; rechaza el correo duplicado, los datos inválidos y el formulario alterado con `role=ADMIN`




Evidencia 3a · Login con el enlace «Crear cuenta» y formulario de registro




Evidencia 3b · Correo ya registrado (409) y cuenta nueva con la sesión abierta como Usuario

## Token expirado

Token expirado (JWT_EXPIRES_IN=3s)

```
$ JWT_EXPIRES_IN=3s npm start

$ POST /api/v1/auth/login  (Ana)
token recibido; vigencia: 3 segundos

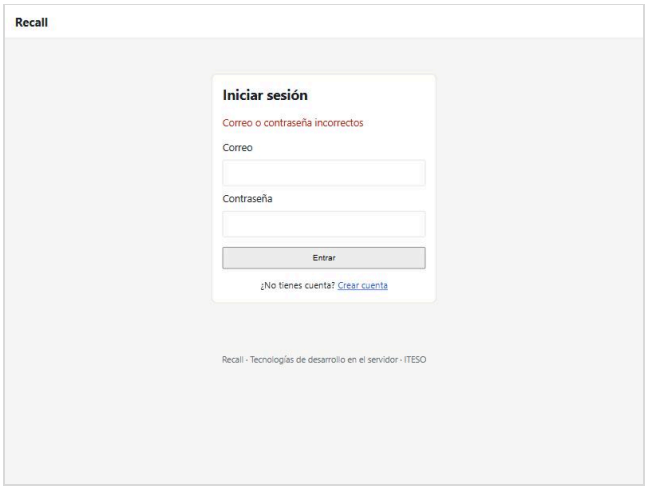
$ GET /api/v1/auth/me  (de inmediato)
[HTTP 200] responde el perfil de Ana

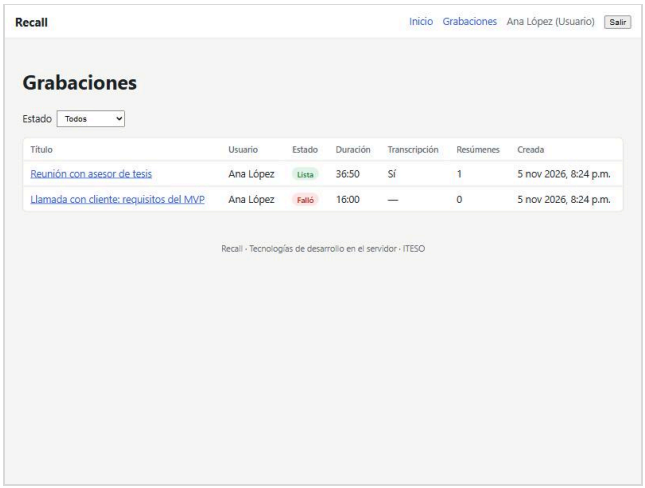
$ sleep 5

$ GET /api/v1/auth/me  (5 s después, mismo token)
{"error":{"message":"Token expired"}} [HTTP 401]
```

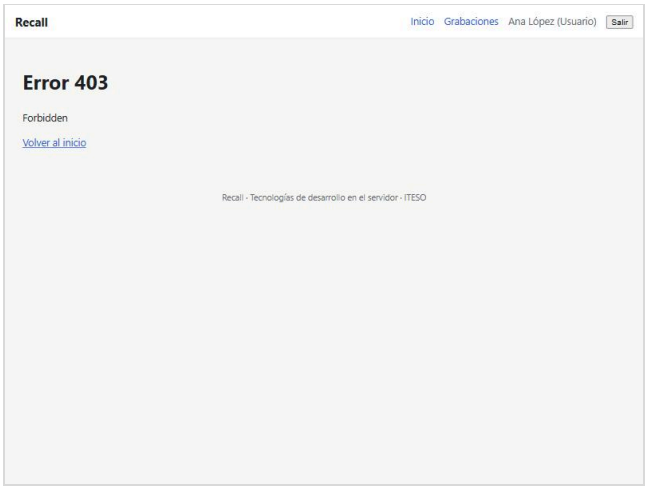
Evidencia 4 · Con `JWT_EXPIRES_IN=3s`, el mismo token funciona al inicio y 5 segundos después responde 401

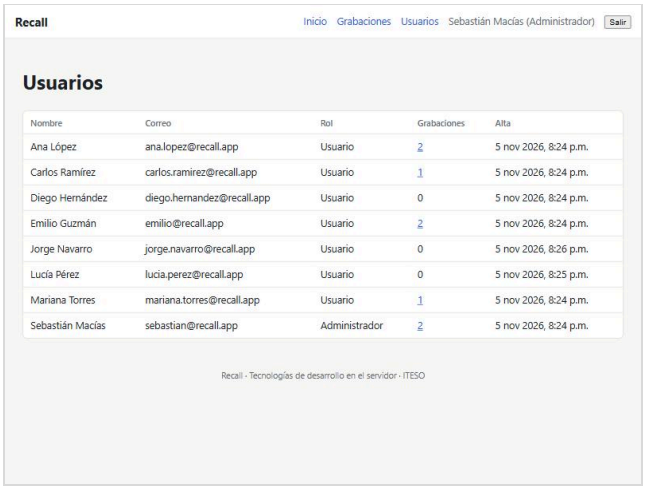
## Login y permisos en las vistas



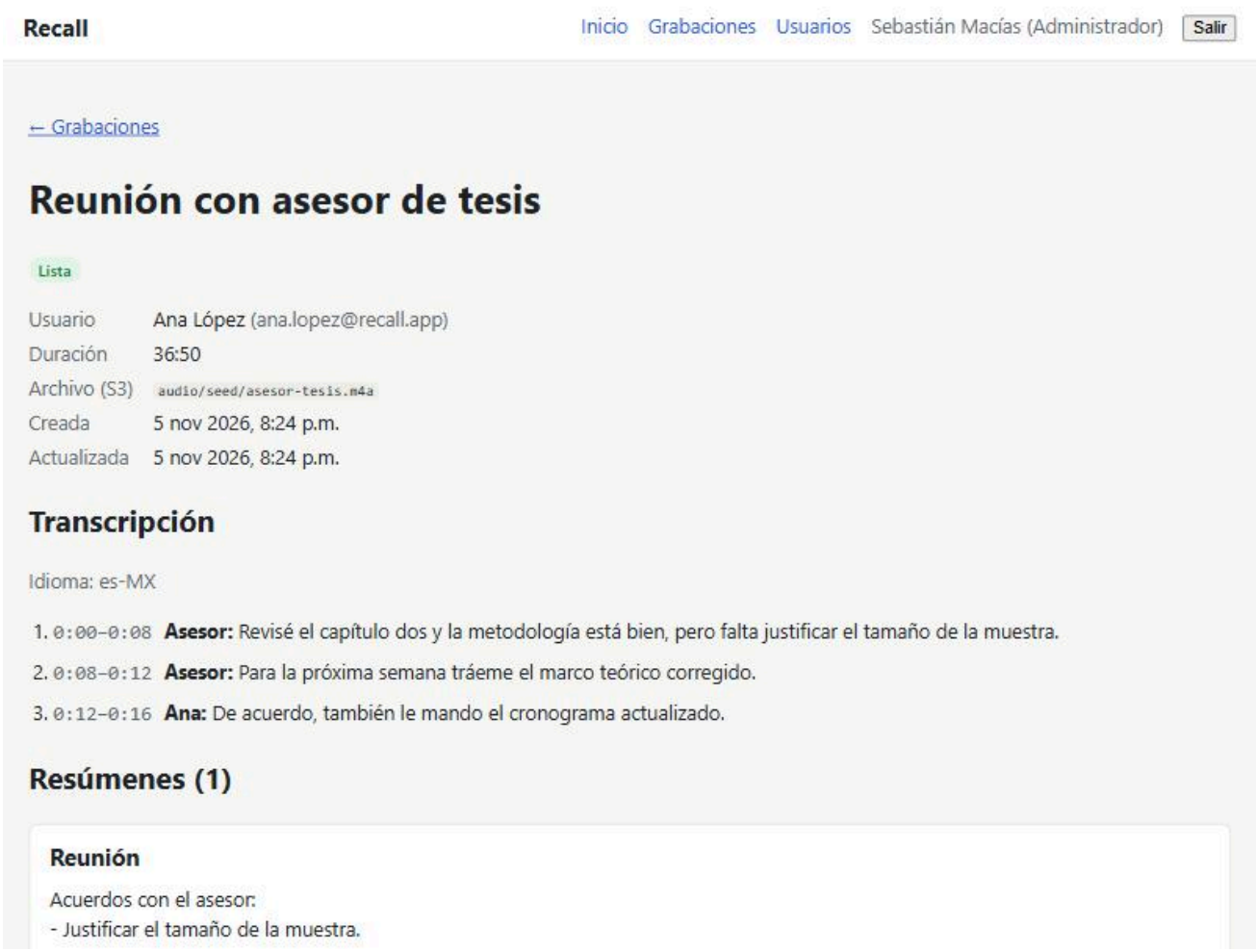




Evidencia 5 · Contraseña incorrecta con mensaje genérico, y Ana López (USER) ve solo sus 2 grabaciones




Evidencia 6 · Ana López recibe 403 en «Usuarios»; Sebastián Macías (ADMIN) ve la lista con los roles




Evidencia 7 · El administrador abre una grabación de Ana López

> **Login con Google:** requiere las credenciales OAuth del proyecto, que no se suben al repositorio. Se prueba en local con `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` y `GOOGLE_CALLBACK_URL` en el `.env` (ver README).

## Requisitos del curso

| Requisito | Cómo lo cubre Recall | Estado |
|---|---|---|
| Base de datos | PostgreSQL 17 con Prisma: 4 tablas relacionadas, índices únicos y migraciones versionadas. | Hecho |
| Validaciones | Middleware `validate` con zod para body, params y query; errores por campo con status 400. | Hecho |
| MVC | Modelos (Prisma + tipos), vistas (Handlebars) y controladores, conectados por rutas por módulo. | Hecho |
| Vistas | Handlebars: registro, login, inicio, listado y detalle de grabaciones, y usuarios, con datos reales y permisos. | Hecho |

| Requisito | Cómo lo cubre Recall | Estado |
|---|---|---|
| Autenticación y permisos | Registro y login propios con bcrypt y JWT; roles USER y ADMIN con matriz de permisos en el API y las vistas. | Hecho (esta entrega) |
| Login con proveedor | Google con OAuth 2.0 / OpenID Connect: vincula por correo verificado o crea un USER. | Hecho (esta entrega) |
| Correos | Amazon SES para avisar al usuario cuando su resumen esté listo, y para verificar el correo al registrarse. | Pendiente |
| Archivos | Subida del audio con multer a un bucket privado de S3; la BD guarda solo `s3Key`. | Entrega 3 (archivos) |
| Servicio de terceros | Amazon Transcribe (transcripción) y Amazon Bedrock (resúmenes), además de Google y S3. | Google hecho; Transcribe y Bedrock pendientes |
| Tiempo real (websockets) | Socket.IO: el API emite el cambio de estado de una grabación (PROCESSING → DONE o FAILED) a la sala del usuario, para que la vista se actualice sin recargar. | Entrega posterior |

## Flujo de trabajo y CI/CD

- Ramas por funcionalidad (por ejemplo `auth_jwt`, `dev_emi_three`) que llegan a `main` por Pull Request revisado por el otro integrante. Nadie trabaja directo en `main`.
- El commit que se evalúa en cada entrega se etiqueta en `main`: `entrega-1` y `entrega-2`.

| Workflow | Qué revisa |
|---|---|
| `ci.yml` | ESLint; `prisma validate`, typecheck y build; con un PostgreSQL de servicio aplica las migraciones, corre el seed dos veces, arranca el API, corre la colección de Postman con newman y revisa que las vistas muestren datos con una sesión real. |
| `sonarqube.yml` | Análisis de SonarQube Cloud (bugs, code smells, duplicación y vulnerabilidades) con quality gate: si no pasa, el workflow falla. |
| `docker.yml` | Construye la imagen del API; en push a `develop` o `main` la publica en GitHub Container Registry. |

## Consideraciones

### Seguridad y privacidad

- Las grabaciones contienen voces y datos personales, por lo que aplica la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP).
- El usuario debe contar con el consentimiento de las personas grabadas.
- Las contraseñas se guardan con bcrypt y `passwordHash` nunca sale en una respuesta. El login no revela qué correos existen.
- `JWT_SECRET`, las credenciales de Google y las de AWS solo viven en el `.env`, que está en `.gitignore`. El `.env.example` trae un `JWT_SECRET` de desarrollo que se rechaza en producción.

- `npm run seed` vacía las tablas: se usa solo en desarrollo.

## Costos

- Google OAuth no tiene costo. S3, Transcribe y Bedrock cobran por uso: se configurará una alerta de AWS Budgets y una duración máxima de audio.
- GitHub Actions y SonarQube Cloud son gratuitos para este uso.

## Próximos pasos

| Entrega | Pendientes |
|---|---|
| Entrega 3: archivos | Subida de audios con multer a S3. |
| Entregas siguientes | Documentación del API con Swagger, transcripción con Amazon Transcribe, resúmenes con Amazon Bedrock, correos con SES, websockets con Socket.IO, paginación y filtros. |